CST2550 Software Engineering Management and Development

Fancy dress hire system

Code Craft members:
Akshita Semwal – M01038587
Mohammed Abdula Afzal – M01036509
Kelly Livoi – M01034045
Ibrahim Wahid – M01010179
Sherine Najmi - M00768217

Contents

Introduction ..... Error! Bookmark not defined.
Design 2
Data Structures 2
1.1 Relational Database Tables 2
1.2 Dynamic Lists 2
1.3 FlowLayoutPanel 2
1.4 UserControl (Card-Based Design) 2
2. Algorithms 3
2.1 Filtering Algorithm 3
2.2 Basket, Orders & Likes Algorithm 3
2.3 Checkout Algorithm 3
2.4 Order Status Update Algorithm 4
2.5 Authentication Algorithm 4
2.6 Due Date Calculation Algorithm 4
Testing and Debugging 4
Table of Test Cases 5
Test Cases 5
Test Type 5
Input 5
Expected Result 5
Actual Result 5
Status 5
Conclusion 6

Introduction

This report presents the design and development of Attire Hire; a fancy dress hire system produced as part of our Software Engineering Management and Development module. The system was created to support users in efficiently browsing and selecting costumes for rental. The system provides a range of core features including browsing available items, adding costumes to a basket, saving items for later through a “like” function, completing the checkout process, and interacting with an integrated AI feature for assistance. The aim of the project is to deliver a clear, user-friendly solution that improves the overall experience of searching for and managing costume hires, while ensuring the system operates reliably and efficiently.

This report is structured into several key sections. The Design section justifies the choice of data structures and algorithms and includes an analysis of the core system functionality using pseudocode. The Testing section outlines the testing approach adopted and presents a table of test cases used to validate the system. The Conclusion summarises the work completed, discusses limitations encountered during development, and reflects on how the approach could be improved in future similar tasks. Finally, the References section lists all sources used, presented in Harvard style with corresponding in-text citations.

Design

Data Structures

The system is designed using a combination of relational database structures and dynamic UI components to ensure efficient data management and scalability.

1.1 Relational Database Tables

A relational database model is used to store and manage system data. The main tables include Clothes, Orders, Basket, Likes, and Users. Primary and foreign keys are used to maintain relationships between entities, such as linking Orders to Clothes using CostumeId. This structure ensures data integrity and supports efficient querying and filtering.

1.2 Dynamic Lists

Dynamic lists are used to store selected filter values such as gender and size. These lists allow flexible addition and removal of elements based on user interaction, making them more suitable than fixed-size arrays.

1.3 FlowLayoutPanel

A FlowLayoutPanel is used to dynamically display items such as clothes, orders, basket items, and liked items. It automatically arranges components and supports runtime updates.

1.4 User Control (Card-Based Design)

User Controls are used to represent individual items such as clothes and orders. This modular approach improves reusability and maintainability, as the same card structure is used across multiple pages.

2. Algorithms

2.1 Filtering Algorithm

The filtering algorithm dynamically constructs a query based on user-selected criteria such as gender, size, and price.

```
BEGIN
CLEAR displayed items
INITIALIZE query as "SELECT all clothes"

IF gender filters selected THEN
ADD selected genders
APPLY condition: Gender IN (selected genders)

IF size filters selected THEN
ADD selected sizes
APPLY condition: Size IN (selected sizes)

APPLY condition: Price <= selected value
EXECUTE query

IF results exist THEN
FOR each item
CREATE card
DISPLAY card
ELSE
DISPLAY "No results found"
END
```

2.2 Basket, Orders & Likes Algorithm

```
BEGIN
WHEN user clicks "Add to Basket"
GET item details

IF item exists in basket, THEN
INCREASE quantity
ELSE
INSERT item

UPDATE cart display
END
```

2.3 Checkout Algorithm

```
BEGIN
WHEN user confirms checkout
FOR each item in basket
```

INSERT into Orders

DELETE basket items
DISPLAY confirmation
END

2.4 Order Status Update Algorithm

BEGIN
SELECT order
UPDATE status

IF notification required THEN
SEND email
END

2.5 Authentication Algorithm

BEGIN
GET user input
VALIDATE input
HASH password

IF match found THEN
GRANT access
ELSE
DISPLAY error
END

2.6 Due Date Calculation Algorithm

BEGIN
GET order date
SET rental duration = 10 days
CALCULATE due date
DISPLAY due date
END

Testing and Debugging

Testing and debugging were essential parts of the development process to ensure the system was reliable, functional, and user-friendly. A combination of debugging techniques, unit testing, integration testing, and manual testing was used throughout the project.

During development, debugging was carried out using Visual Studio tools such as breakpoints, the Watch window, and Message Box outputs. These techniques helped identify and resolve issues such as incorrect data loading, null values, and data type mismatches (for example, handling decimal values for

prices). Exception handling using try-catch blocks was also implemented to prevent system crashes and provide meaningful error messages to users.

For automated testing, MS Test was used as the primary testing framework. Unit testing was applied to validate core logic within the system, including price calculations, shipping conditions, and filtering query construction. To enable effective unit testing, key logic (such as filtering functionality) was separated into helper classes, allowing it to be tested independently from the user interface.

Integration testing was also performed to verify interactions between the application and the database. Tests were conducted to ensure successful database connectivity and correct execution of operations such as inserting, retrieving, and deleting data from key tables including Clothes, Basket, Likes, and Orders. These tests ensured data integrity and confirmed that the system interacts correctly with the database.

In addition, end-to-end testing was carried out manually to simulate real user workflows. This included adding items to the basket, selecting items for checkout, placing orders, and updating order statuses. Email notifications were also tested to ensure that customers receive updates when their order status changes.

Overall, the combination of debugging, unit testing using MS Test, integration testing, and manual testing ensured that the system functions correctly, handles errors gracefully, and meets user requirements.

The system includes an AI component using Semantic Kernel and OpenAI to allow users to search using natural language. The AI interprets user input, extracts key details such as gender and price, and uses these to query the database. It also generates human-like responses to guide the user. This enhances usability by providing an intelligent and interactive search experience.

Table of Test Cases

Test Cases	Test Type	Input	Expected Result	Actual Result	Status
Add to basket	Manual	User clicks “Add”	Item is added to basket	Item added successfully	Pass
Checkout with selection	Manual	User selects specific items and clicks checkout	Only selected items appear in the checkout grid	Correct items displayed	Pass
Checkout without selection	Manual	User clicks without selecting items	All basket items appear	All items displayed	Pass

Price Filter	Unit / Manual	User sets max price using slider	Only items ≤ selected price are shown	Filter works correctly	Pass
Search Function	Unit/ Manual	User enters search term	Matching items are displayed	Correct results displayed	Pass
Filtering Logic	Unit (MS Test)	Apply gender, size, and search filters	Correct SQL query is generated	Query generated correctly	Pass
Database Connection	Integration (MS Test)	System attempts DB connection	Connection opens successfully	Connection successfully	Pass
Insert Order, Liked, Basket	Integration (MS Test)	Insert test order, Liked, Basket into database	Order is added to Orders, Liked, Basket table	Insertion successful	Pass
Place order	Manual	User clicks “Order”	Order saved and basket cleared	Order created successfully	Pass
Email Notification	Manual	Admin updates order status	Email sent to customer	Email sent successfully	Pass
Invalid Payment input	Manual	User leaves payment fields empty	Error message displayed	Validation message shown	Pass
UX Design	Manual	Showed other students the design to get feedback	Positive Feedback	Positive feedback	Pass
AI Search 1	Manual	Random text	AI Responds	Connection successful	Pass
AI Search 2	Manual	Cheap costumes	Returns low price items	Connection successful	Passed
AI Search 3	Manual	Male costumes under 30	Filters male + price	Successful	Pass

Conclusion

In conclusion, the Attire Hire system was successfully designed and developed as a functional fancy dress hire platform, incorporating key features such as browsing items, adding products to a basket, saving items for later, completing the checkout process, and interacting with an AI support feature. The

system successfully fulfils its intended purpose by offering a straightforward and user-friendly way to manage fancy dress hire, reflecting the group's ability to develop a functional and coherent application.

However, several limitations were identified throughout the development process. One of the main challenges was maintaining effective communication outside of scheduled sessions. Although meetings were conducted, ensuring consistent updates and coordination beyond these proved difficult, particularly as one group member was in a different class. This occasionally led to delays in decision-making and some misalignment between different parts of the system.

As a group, balancing responsibilities between front-end and back-end development alongside managing overall progress was also challenging. At times, team members worked at different speeds, which made it difficult to keep all aspects of the project aligned with the intended timeline. In addition, limited information gathering and planning at the early stages of the project meant that some requirements were not fully understood from the outset. This resulted in parts of the system needing to be revisited and adjusted during development, which increased workload and reduced overall efficiency.

The allocation of roles within the team also presented some challenges. While responsibilities were generally assigned, there were instances where tasks overlapped or lacked clear ownership, leading to some inefficiencies. This highlighted the importance of clearly defined roles and stronger coordination throughout the development process.

From a technical perspective, some features were more challenging to implement than originally anticipated. Developing the functionality of the "like" feature presented difficulties, as ensuring that selected items were correctly updated and displayed required careful handling of data across different parts of the system. This emphasised the challenge of ensuring that user actions were accurately processed and reflected across the system, especially when handling data between different components. Additionally, designing the homepage was challenging, as it required balancing visual clarity, organisation, and consistency across the interface.

Despite these challenges, the project provided valuable learning opportunities. It improved the group's understanding of how front-end and back-end components interact, as well as the importance of planning, communication, and coordination within a team-based development environment. With additional time, the group would aim to refine the layout and organisation of the homepage, alongside improving the consistency and usability of the like feature to enhance the overall user experience. Furthermore, placing greater emphasis on early planning, clearer role definition, and more consistent communication would contribute to a more efficient development process and a higher-quality final system.